

Synthesis in Linear Temporal Logics on Finite Traces

Game-theoretic Approach to LTLf Planning and Synthesis

Giuseppe De Giacomo



Program Synthesis

- **Basic Idea:** “Mechanical translation of human-understandable task specifications to a program that is known to meet the specifications.” [Vardi - The Siren Song of Temporal Synthesis 2018]
- Classical vs. Reactive Synthesis:
 - ▶ Classical: Synthesize transformational programs [Green1969], [WaldingerLee1969], [Manna and Waldinger1980]
 - ▶ **Reactive:** Synthesize programs for interactive/reactive ongoing computations (protocols, controllers, robots, etc.) [Church1963], [AbadiLamportWolper1989], [PnueliRosner1989]

Reactive Synthesis

- Reactive synthesis is equipped with a elegant and comprehensive theory [Finkbeiner2018],[EhlersLafortuneTripakisVardi2017]
- Reactive synthesis is conceptually related to planning in nondeterministic domains [DeGiacomoVardi2015], [DeGiacomoRubin2018], [CamachoMuiseBaierMcIlraith2018]

Program Synthesis

- **Basic Idea:** “Mechanical translation of human-understandable task specifications to a program that is known to meet the specifications.” [Vardi - The Siren Song of Temporal Synthesis 2018]
- Classical vs. Reactive Synthesis:
 - ▶ Classical: Synthesize transformational programs
[Green1969], [WaldingerLee1969], [Manna and Waldinger1980]
 - ▶ **Reactive:** Synthesize programs for interactive/reactive ongoing computations (protocols, controllers, robots, etc.)
[Church1963], [AbadiLamportWolper1989], [PnueliRosner1989]

Reactive Synthesis

- Reactive synthesis is equipped with a elegant and comprehensive theory
[Finkbeiner2018],[EhlersLafortuneTripakisVardi2017]
- Reactive synthesis is conceptually related to planning in nondeterministic domains
[DeGiacomoVardi2015], [DeGiacomoRubin2018], [CamachoMuiseBaierMcIlraith2018]



Inputs and outputs

- The agent receives input x from the environment.
- The agent sends output y to the environment.
- Input x can be fluents, features, program input, etc.
- Output y can be actions, control instructions, program outputs, etc.
- Input is uncontrollable by the agent (it is under the control of the environment).
- Output is controllable by the agent.



Game View

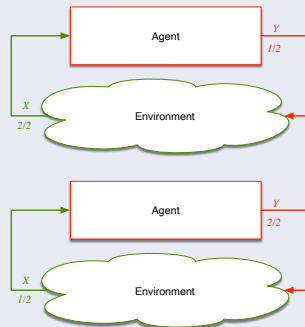
Agent is playing a game with **environment**, with the LTL_f / LTL specifications being the winning condition.

- **Agent** chooses **controllable** output $Y \in 2^{\mathcal{Y}}$
- **Environment** chooses **uncontrollable** input $X \in 2^{\mathcal{X}}$
- **Round**: **agent** and **environment** set their values
- **Play**: finite trace τ over $(\mathcal{X} \cup \mathcal{Y})$
- **Agent** decides when to stop
- **Specification**: LTL_f formula φ
- **Agent** wins $\tau \models \varphi$

Game Rounds

Agent is playing a game with **environment**, with the LTL_f / LTL specifications being the winning condition.

- **Agent** chooses **controllable** output $Y \in 2^{\mathcal{Y}}$
- **Environment** chooses **uncontrollable** input $X \in 2^{\mathcal{X}}$
- **Round:** **agent** and **environment** set their values
 - ▶ Pair **output** and resulting **input**
(**agent action** and **environment reaction**)
 - ▶ Pair **input** and next output
(**environment state** and **next agent action**)
- **Play:** finite trace τ over $(\mathcal{X} \cup \mathcal{Y})$
- **Agent** decides when to stop
- **Specification:** LTL_f formula φ
- **Agent** wins $\tau \models \varphi$



Pair actions and states in a time instant (reminder)

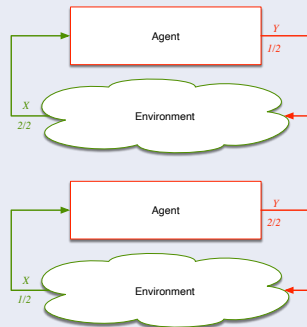
Decide how we need to pair actions and states in a time instant

- Pair the agent **action** and the **resulting state**, (in fact labeling of the state) of the environment
The propositional representation a for an action a will stand for "**action a just executed**".
- Pair current environment (labeling of the) **state** and the **next action** instructed by the agent
The propositional representation a for an action a will stand for "**action a just instructed to be executed next**".

Game Rounds

Agent is playing a game with **environment**, with the LTL_f / LTL specifications being the winning condition.

- **Agent** chooses **controllable** output $Y \in 2^{\mathcal{Y}}$
- **Environment** chooses **uncontrollable** input $X \in 2^{\mathcal{X}}$
- **Round:** **agent** and **environment** set their values
 - ▶ Pair **output** and resulting **input** $\Leftarrow$
(**agent action** and **environment reaction**)
 - ▶ Pair **input** and next **output**
(**environment state** and **next agent action**)
- Play: finite trace τ over $(\mathcal{X} \cup \mathcal{Y})$
- **Agent** decides when to stop
- Specification: LTL_f formula φ
- **Agent** wins $\tau \models \varphi$



Pair actions and states in a time instant (reminder)

Decide how we need to pair actions and states in a time instant

- Pair the agent **action** and the **resulting state**, (in fact labeling of the state) of the environment $\Leftarrow$
The propositional representation a for an action a will stand for "**action a just executed**".
- Pair current environment (labeling of the) **state** and the **next action** instructed by the agent
The propositional representation a for an action a will stand for "**action a just instructed to be executed next**".



Agent strategies

Agent strategy (*also called, “plan”, “policy”, “protocol”, “process”, “program”, “behavior”*):

$$\sigma_a : (2^{\mathcal{X}})^* \rightarrow 2^{\mathcal{Y}}$$

where

- $(2^{\mathcal{X}})^*$ denotes the **history** of inputs observed so far by the agent

(a finite sequence of fluents configurations)

- $2^{\mathcal{Y}}$ denotes the **next output** of the agent

Every program/process has this form! [AbadiLamportWolper89].

Game View

Agent is playing a game with **environment**, with the LTL_f/LTL specifications being the **winning condition**.

- **Agent** chooses **controllable** output $Y \in 2^{\mathcal{Y}}$
- **Environment** chooses **uncontrollable** input $X \in 2^{\mathcal{X}}$
- **Round**: **agent** and **environment** set their values
- **Play**: finite trace τ over $(\mathcal{X} \cup \mathcal{Y})$
- **Agent** decides when to stop
- **Specification**: LTL_f formula φ
- **Agent** wins $\tau \models \varphi$

Realizability and Synthesis

- **Agent strategy** σ_a is **winning** if for every infinite sequence $X_0, X_1, X_2 \dots$ the play $(\sigma_a(\epsilon), X_0)(\sigma_a(X_0), X_1)(\sigma_a(X_0, X_1), X_2) \dots$ there exists an instant n such that at the trace $\tau = (\sigma_a(\epsilon), X_0), \dots, (\sigma_a(X_0, \dots, X_{n-1}), X_n)$ satisfies φ , i.e., $\tau \models \varphi$.
- **Realizability**: **exists** a winning agent strategy σ_a .
- **Synthesis**: **obtain** a winning agent strategy σ_a .

Game View

Agent is playing a game with **environment**, with the LTL_f/LTL specifications being the **winning condition**.

- **Agent** chooses **controllable** output $Y \in 2^{\mathcal{Y}}$
- **Environment** chooses **uncontrollable** input $X \in 2^{\mathcal{X}}$
- **Round**: **agent** and **environment** set their values
- **Play**: finite trace τ over $(\mathcal{X} \cup \mathcal{Y})$
- **Agent** decides when to stop
- **Specification**: LTL_f formula φ
- **Agent** wins $\tau \models \varphi$

Realizability and Synthesis

- **Agent strategy** σ_a is **winning** if for every infinite sequence $X_0, X_1, X_2 \dots$ the play $(\sigma_a(\epsilon), X_0)(\sigma_a(X_0), X_1)(\sigma_a(X_0, X_1), X_2) \dots$ there exists an instant n such that at the trace $\tau = (\sigma_a(\epsilon), X_0), \dots, (\sigma_a(X_0, \dots, X_{n-1}), X_n)$ satisfies φ , i.e., $\tau \models \varphi$.
- **Realizability**: **exists** a winning agent strategy σ_a .
- **Synthesis**: **obtain** a winning agent strategy σ_a .

Synthesis from LTL_f Specifications

- Specify **task with LTL_f formulas**
- Relay on transformation of LTL_f formulas into DFA
- Solve game on the DFA: find an **agent strategy** to reach a final state in spite of how the **environment** reacts.

Synthesis from LTL_f Specifications

Given a LTL_f formula φ over a set $\mathcal{P}$ of propositions partitioned into two disjoint sets:

- $\mathcal{X}$ controlled by **environment**
- $\mathcal{Y}$ controlled by **agent**

Find an **agent strategy** σ_a to set the values of $\mathcal{Y}$ in such a way that for all possible values of $\mathcal{X}$, controlled by the **environment**, the LTL_f formula φ can be made true.

Algorithm for LTL_f synthesis

- 1: Given LTL_f formula φ
- 2: Compute AFA for φ (linear)
- 2: Compute corresponding NFA (exponential)
- 3: Determinize NFA to DFA (exponential)
- 4: Synthesize winning strategy for DFA game (linear)
- 5: Return strategy as a trasducer

Thm: LTL_f synthesis is 2-EXPTIME-complete.

Same as for infinite traces

Synthesis from LTL_f Specifications

Given a LTL_f formula φ over a set $\mathcal{P}$ of propositions partitioned into two disjoint sets:

- $\mathcal{X}$ controlled by **environment**
- $\mathcal{Y}$ controlled by **agent**

Find an **agent strategy** σ_a to set the values of $\mathcal{Y}$ in such a way that for all possible values of $\mathcal{X}$, controlled by the **environment**, the LTL_f formula φ can be made true.

Algorithm for LTL_f synthesis

- 1: Given LTL_f formula φ
- 2: Compute AFA for φ (**linear**)
- 2: Compute corresponding NFA (**exponential**)
- 3: Determinize NFA to DFA (**exponential**)
- 4: Synthesize winning strategy for DFA game (**linear**)
- 5: Return strategy as a trasducer

Thm: LTL_f synthesis is 2-EXPTIME-complete.

Same as for infinite traces

DFA games

A DFA game $\mathcal{G} = (2^{\mathcal{X} \cup \mathcal{Y}}, S, s_0, \delta, F)$, is such that:

- $\mathcal{X}$ controlled by **environment**; $\mathcal{Y}$ controlled by **agent**;
- $2^{\mathcal{X} \cup \mathcal{Y}}$, alphabet of game;
- S , states of game;
- s_0 , initial state of game;
- $\delta : S \times 2^{\mathcal{X} \cup \mathcal{Y}} \rightarrow S$, transition function of the game: given current state s and a choice of propositions X and Y the resulting state of the game is $\delta(s, (X, Y)) = s'$;
- F , final states of game, where game can be considered terminated.

Winning condition for DFA games

Let

$$PreAdv(\mathcal{E}) = \{s \in S \mid \exists Y \in 2^{\mathcal{Y}}. \forall X \in 2^{\mathcal{X}}. \delta(s, (X, Y)) \in \mathcal{E}\}$$

Compute the set $Win(\mathcal{G})$ of winning states of a DFA game $\mathcal{G}$, i.e., states from which the agent can win the DFA game $\mathcal{G}$, by least-fixpoint:

- $Win_0(\mathcal{G}) = F$ (the final states of $\mathcal{G}$)
- $Win_{i+1}(\mathcal{G}) = Win_i(\mathcal{G}) \cup PreAdv(Win_i(\mathcal{G}))$
- $Win(\mathcal{G}) = \bigcup_i Win_i(\mathcal{G})$

Computing $Win(\mathcal{G})$ is *linear* in the number of states in $\mathcal{G}$.

DFA games

A DFA game $\mathcal{G} = (2^{\mathcal{X} \cup \mathcal{Y}}, S, s_0, \delta, F)$, is such that:

- $\mathcal{X}$ controlled by **environment**; $\mathcal{Y}$ controlled by **agent**;
- $2^{\mathcal{X} \cup \mathcal{Y}}$, alphabet of game;
- S , states of game;
- s_0 , initial state of game;
- $\delta : S \times 2^{\mathcal{X} \cup \mathcal{Y}} \rightarrow S$, transition function of the game: given current state s and a choice of propositions X and Y the resulting state of the game is $\delta(s, (X, Y)) = s'$;
- F , final states of game, where game can be considered terminated.

Winning condition for DFA games

Let

$$PreAdv(\mathcal{E}) = \{s \in S \mid \exists Y \in 2^{\mathcal{Y}}. \forall X \in 2^{\mathcal{X}}. \delta(s, (X, Y)) \in \mathcal{E}\}$$

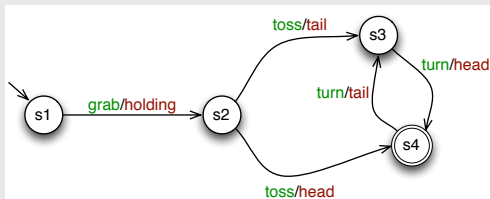
Compute the set $Win(\mathcal{G})$ of winning states of a DFA game $\mathcal{G}$, i.e., states from which the agent can win the DFA game $\mathcal{G}$, by least-fixpoint:

- $Win_0(\mathcal{G}) = F$ (the final states of $\mathcal{G}$)
- $Win_{i+1}(\mathcal{G}) = Win_i(\mathcal{G}) \cup PreAdv(Win_i(\mathcal{G}))$
- $Win(\mathcal{G}) = \bigcup_i Win_i(\mathcal{G})$

Computing $Win(\mathcal{G})$ is *linear* in the number of states in $\mathcal{G}$.

Example (Toss a coin)

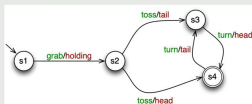
Consider the following (very simple) DFA game. Where the agent can **grab** a coin, **toss** it and **turn** it and the environment responds to grab with the deterministic effect **holding**, to toss by **tail or head** (devilish nondeterminism), and to turn by (deterministically) **changing the coin side**. The goal of the game is to choose appropriately **grab**, **toss**, and **turn** to get **head** in the hand.



Example of DFA Game

Example (Toss a coin)

Consider the following (very simple) DFA game. Where the agent can **grab** a coin, **toss** it and **turn** it and the environment responds to grab with the deterministic effect **holding**, to toss by **tail** or **head** (devilish nondeterminism), and to turn by (deterministically) **changing the coin side**. The goal of the game is to choose appropriately grab, toss, and turn to get **head** in the hand.



Example (Toss a coin)

Compute the winning set

- $Win_0 = \{s_4\}$ (the final states of the game)
- $Win_1 = Win_0 \cup \{s \in S \mid \exists Y \in 2^Y. \forall X \in 2^X. \delta(s, (X, Y)) \in Win_0\} = \{s_4\} \cup \{s_3\}$
- $Win_2 = Win_1 \cup \{s \in S \mid \exists Y \in 2^Y. \forall X \in 2^X. \delta(s, (X, Y)) \in Win_1\} = \{s_3, s_4\} \cup \{s_2\}$
- $Win_3 = Win_2 \cup \{s \in S \mid \exists Y \in 2^Y. \forall X \in 2^X. \delta(s, (X, Y)) \in Win_2\} = \{s_2, s_3, s_4\} \cup \{s_1\}$

So the agent win from all states!

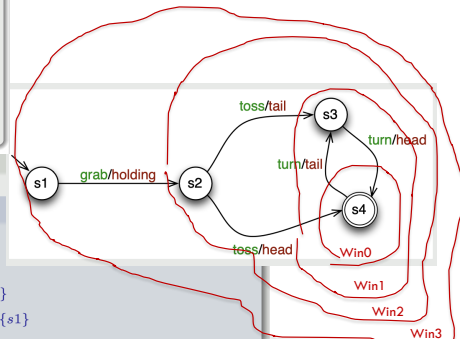
Compute the strategy generator

In fact it is necessary to compute only the output function ω (the rest of the trasducer is determined by such an ω):

$$\omega(s) = \{Y \mid \text{if } s \in Win_{i+1}(\mathcal{G}) - Win_i(\mathcal{G}) \text{ then } \forall X. \delta(s, (X, Y)) \in Win_i(\mathcal{G})\}.$$

In our case:

$$\begin{aligned} \omega(s_1) &= \{\text{grab}\} \\ \omega(s_2) &= \{\text{toss}\} \\ \omega(s_3) &= \{\text{turn}\} \\ \omega(s_4) &= \text{WIN} \quad \text{it is the goal state!} \end{aligned}$$



To actually compute a strategy, we need to

- Apply any choice function to get only one value $choice(\omega(s))$ (any choice would be good) among those in $\omega(s)$, where

$$\omega(s) = \{Y \mid \text{if } s \in Win_{i+1}(\mathcal{G}) - Win_i(\mathcal{G}) \text{ then } \forall X. \delta(s, (X, Y)) \in Win_i(\mathcal{G})\}$$

- Compute the corresponding strategy $\sigma_a : (2^{\mathcal{X}})^* \rightarrow 2^{\mathcal{Y}}$ via a transducer $\mathcal{T}_{\mathcal{G}}$ obtained from the game $\mathcal{G}$ and the function $choice(\omega(s))$.
The obtained σ_a is memory-full, but has only finite number of states.

Strategy as a transducer

Given the DFA game $\mathcal{G} = (2^{\mathcal{X} \cup \mathcal{Y}}, S, s_0, \delta, F)$ and the function $choice(\omega(s))$, the strategy returned is a transducer

$$\mathcal{T}_{\mathcal{G}} = (2^{\mathcal{X}}, S, s_0, \varrho, \omega_{choice})$$

where:

- $2^{\mathcal{X}}$ is the alphabet of the transducer;
- S are the states of the transducer;
- s_0 is the initial state;
- $\varrho : S \times 2^{\mathcal{X}} \rightarrow S$ is the transition function (partial) such that $\varrho(s, X) = \delta(s, (X, choice(\omega(s))))$
- $\omega_{choice} : S \rightarrow 2^{\mathcal{Y}}$ is the output function such that $\omega_{choice} = choice(\omega(s))$

Synthesis for general LTL specifications **does not scale (yet)**.

Solving reactive synthesis

Algorithm for LTL synthesis

Given LTL formula φ

- 1: Compute corresponding Buchi Nondeterministic Aut. (NBW) (**exponential**)
- 2: Determinize NBW into Deterministic parity Aut. (DPW) (**exp in states, poly in priorities**)
- 3: Synthesize winning strategy for **parity game** (**poly in states, exp in priorities**)

Return strategy

Reactive synthesis is 2EXPTIME-complete, but more importantly the **problems are**:

- The **determinization** in Step 2: **no scalable algorithm exists for it yet**.
 - ▶ From 9-state NBW to 1,059,057-state DRW [AlthoffThomasWallmeier2005]
 - ▶ No symbolic algorithms
- Solving **parity games** requires computing **nested fixpoints** (possibly exp many)

Synthesis for general LTL specifications **does not scale (yet)**.

Solving reactive synthesis

Algorithm for LTL synthesis

Given LTL formula φ

- 1: Compute corresponding Buchi Nondeterministic Aut. (NBW) (**exponential**)
- 2: Determinize NBW into Deterministic parity Aut. (DPW) (**exp in states, poly in priorities**)
- 3: Synthesize winning strategy for **parity game** (**poly in states, exp in priorities**)

Return strategy

Reactive synthesis is 2EXPTIME-complete, but more importantly the **problems are**:

- The **determinization** in Step 2: **no scalable algorithm exists for it yet**.
 - ▶ From 9-state NBW to 1,059,057-state DRW [AlthoffThomasWallmeier2005]
 - ▶ No symbolic algorithms
- Solving **parity games** requires computing **nested fixpoints** (possibly exp many)